AmpereHour energy Golang System Developer - What to Learn

MUST Learn (Class A keywords with no direct project experience)

- **Go (Golang)** – the JD's primary language, and your production experience is entirely JavaScript/TypeScript/Pyt Go compiles to native binaries, uses goroutines/channels for concurrency instead of async/await, and has no garbage-collector-free memory model like C++/Rust but is still much closer to systems programming than Node.js. Given this JD's low-level focus, plan to build a small terminal or gRPC-based service in Go, not just a REST API.
- **Systems programming fundamentals (OS, memory management, multithreading/multiprocessing)** – your background is application-level (React/Node), not systems-level. Core topics to study: process vs. thread, virtual memory, the difference between multiprocessing (separate memory spaces) and multithreading (shared memory) and the synchronization primitives each requires (mutexes, semaphores).
- **GUI programming for desktop/terminal applications** – this JD wants both GUI and terminal-app development, which is a different paradigm from React's declarative component model. For Go specifically, look at Fyne or Gio for GUI, and libraries like tview/bubbletea for terminal UIs (TUIs).
- **Industrial/IoT protocols (Modbus, OPC)** – these are specialized communication protocols used in industrial and energy systems (fitting, given AmpereHour's energy-sector focus) with no analogue in your web/mobile experience. Modbus is a simple serial/TCP protocol for talking to industrial devices; OPC (OPC-UA) is a more modern, structured protocol for the same space. This is likely the single most novel area for you – budget real time here if you pursue this role.

GOOD to Learn (strengthens the application, not blocking)

- **gRPC and MQTT, applied** – you understand REST API design well; gRPC (binary, contract-first via protobuf) and MQTT (lightweight pub/sub for IoT) are different communication patterns worth a hands-on tutorial each, since they're both explicitly named in the JD.
- **C/C++ or Rust, refreshed** – you have C/C++ CS-fundamentals background (CodeChef, coursework); refresh pointer/memory semantics specifically, since this JD's "compiled languages" framing implies comfort with manual memory management, which Go itself abstracts away but interviewers may still probe.

Fast-Ramp Note

This JD sits further from your current stack than most you've tailored for – it's systems/embedded-adjacent (OS, networking, industrial protocols, GUI) rather than web/API development. Your strongest transferable assets are DSA/algorithms fundamentals (6-star CodeChef, 1000+ problems) and REST API design experience – everything else here (Go itself, Modbus/OPC, GUI programming, deep OS/memory concepts) is genuinely new ground. If you pursue this role, treat it as a deliberate pivot into systems programming rather than an incremental stretch.